

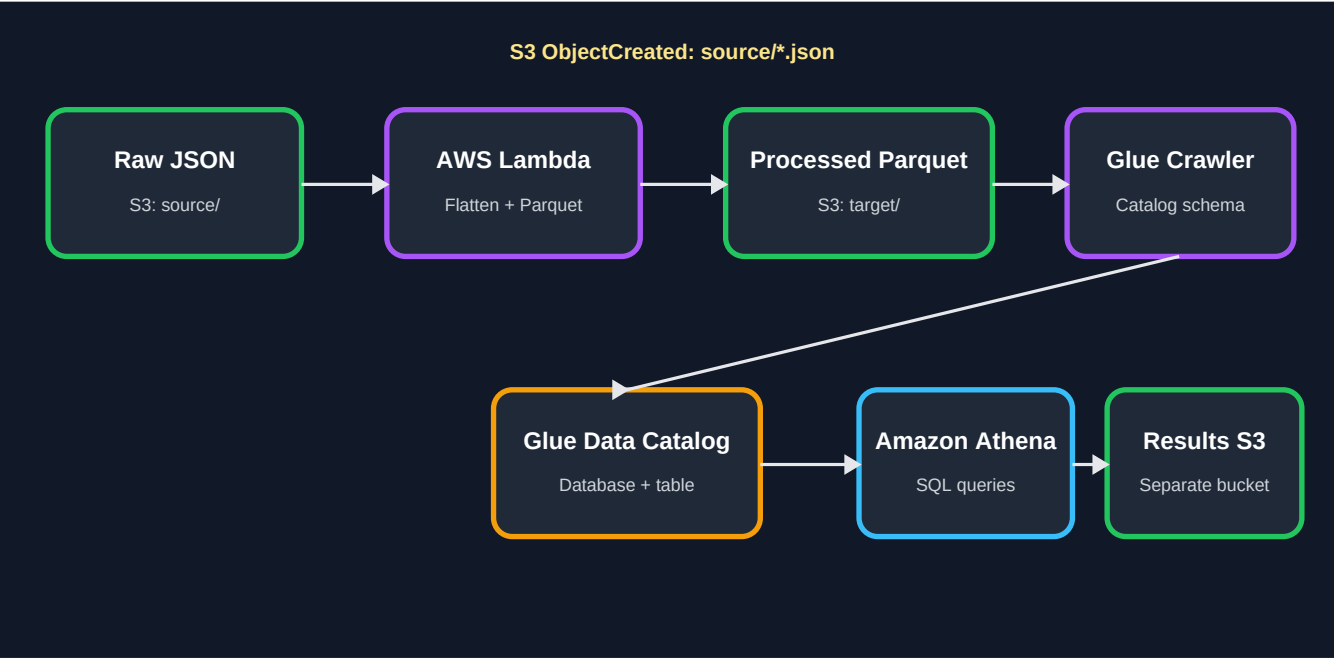
AWS Serverless Event-Driven JSON-to-Parquet ETL Pipeline

Complete A-to-Z implementation guide: Amazon S3, AWS Lambda, Pandas/PyArrow layer, AWS Glue Crawler, Glue Data Catalog, Amazon Athena, IAM, testing, troubleshooting, and cleanup.

Goal: Upload nested JSON orders into an S3 source folder. S3 invokes Lambda automatically. Lambda flattens nested products, writes Apache Parquet to an S3 target folder, starts a Glue crawler, and Athena queries the cataloged output.

Architecture diagram

This is the intended final architecture. It uses one data-lake bucket with separate source and target prefixes, plus a separate bucket for Athena query results.



Critical separation: Athena query-result CSV files must never be stored in the processed Parquet prefix. Use the separate results bucket shown above.

1. Final design

Component	Purpose	Example
S3 data-lake bucket	Stores raw source JSON and processed Parquet in separate prefixes	yourname-athena-query-results
S3 source prefix	Receives uploaded nested JSON	source/orders.json
AWS Lambda	Runs automatically when a JSON object is created; flattens JSON and writes Parquet	etl_pipeline_lambda
S3 target prefix	Stores only Parquet data that Glue/Athena will read	target/orders_parquet_datalake/
AWS Glue crawler	Discovers Parquet schema and creates/updates table metadata	etl_pipeline_crawler
Glue Data Catalog	Stores database/table metadata used by Athena	etl_pipeline_db
Amazon Athena	Runs SQL directly on Parquet in S3	SELECT ... LIMIT 10
Athena results bucket	Stores Athena query output separately	yourname-athena-query-results

2. Names, Region, prerequisites

Use a single AWS Region for all services. For India, you may choose **ap-south-1 (Mumbai)**. Bucket names must be globally unique, so replace *yourname* below.

```
Data-lake bucket: yourname-etl-data-lake
Athena results bucket: yourname-athena-query-results
Lambda function: json-to-parquet-lambda
Lambda role: json-to-parquet-lambda-role
Glue database: etl_pipeline_db
Glue crawler: etl_pipeline_crawler
Crawler role: etl-pipeline-crawler-role
```

- You need permission in your AWS account to create S3 buckets, Lambda functions/layers, IAM roles/policies, Glue resources, and Athena queries.
- Keep “Block all public access” enabled for both S3 buckets.
- Use small JSON files for this Lambda learning project. For very large files or high-volume batch processing, use AWS Glue ETL/Spark instead.

3. Create S3 buckets

3.1 Data-lake bucket

In Amazon S3, choose **Create bucket**, create **yourname-etl-data-lake** in your selected Region, keep all public access blocked, then create these prefixes by choosing Create folder:

```
source/
target/
target/orders_parquet_datalake/
```

3.2 Athena results bucket

Create a second S3 bucket named **yourname-athena-query-results** in the same Region. Create this prefix:

```
athena-query-results/
```

Required layout:

```
yourname-etl-data-lake/
■■■ source/ # raw JSON only
■■■ target/
■■■ orders_parquet_datalake/ # Parquet only

yourname-athena-query-results/
■■■ athena-query-results/ # Athena result CSV/metadata only
```

4. Configure Athena result location

Open **Amazon Athena** → **Settings** → **Manage**. Set the query result location to:

```
s3://yourname-athena-query-results/athena-query-results/
```

Save the setting. If your Athena workgroup overrides client settings, configure the same result location under **Athena** → **Workgroups** → **primary** → **Edit**.

Do this before running SQL. Athena needs a results location. The separate results bucket prevents CSV query output from being scanned as Parquet by the crawler/table.

5. Create Lambda execution role

Create a role in **IAM** → **Roles** → **Create role**: trusted entity = AWS service; service = Lambda; role name = **json-to-parquet-lambda-role**. Attach the managed policy:

```
AWSLambdaBasicExecutionRole
```

Then add this inline policy. Replace **YOUR-DATA-LAKE-BUCKET** with your actual data-lake bucket name.

```
{
  "Version": "2012-10-17",
  "Statement": [
```

```
{
  "Sid": "ReadJsonFromSourceFolder",
  "Effect": "Allow",
  "Action": ["s3:GetObject"],
  "Resource": "arn:aws:s3:::YOUR-DATA-LAKE-BUCKET/source/*"
},
{
  "Sid": "WriteParquetToTargetFolder",
  "Effect": "Allow",
  "Action": ["s3:PutObject"],
  "Resource": "arn:aws:s3:::YOUR-DATA-LAKE-BUCKET/target/orders_parquet_datalake/*"
},
{
  "Sid": "StartGlueCrawler",
  "Effect": "Allow",
  "Action": ["glue:StartCrawler", "glue:GetCrawler"],
  "Resource": "*"
}
}
```

For a learning project, crawler resource `*` is acceptable. In production, restrict it to the specific crawler ARN.

6. Lambda layer: Pandas + PyArrow

Your code uses both Pandas and PyArrow. A Pandas-only layer is not enough because Parquet output requires PyArrow.

```
import pandas as pd
...
df.to_parquet(parquet_buffer, index=False, engine="pyarrow")
```

Your layer must include:

```
pandas
pyarrow
numpy
```

The ZIP must contain a root-level **python/** directory:

```
layer.zip
├── python/
├── pandas/
├── pyarrow/
├── numpy/
└── *.dist-info/
```

- Build the layer for Amazon Linux/Lambda, not directly from Windows or macOS unless you use a compatible build environment.
- The layer Python runtime must match the Lambda runtime, e.g., Python 3.12 with Python 3.12.
- The layer architecture must match Lambda: x86_64 with x86_64, or arm64 with arm64.
- Attach the layer in Lambda → your function → Layers → Add a layer → Custom layers.

7. Create Lambda function

In **AWS Lambda** → **Create function**:

Setting	Value
Function name	json-to-parquet-lambda
Runtime	Same Python version as the layer
Architecture	Same architecture as the layer
Execution role	Use existing role: json-to-parquet-lambda-role
Memory	1024 MB (good starting point for Pandas/PyArrow)
Timeout	1 minute (increase if needed)

After creating it, attach your Pandas + PyArrow layer and click **Deploy** after adding the code in the next step.

8. Lambda code

This code supports the one-bucket design. It reads only JSON source events, flattens each product into one row, writes a Parquet file into target/, and starts the crawler after successful output.

```
import json
import boto3
import pandas as pd
import io
import urllib.parse
from datetime import datetime

def flatten(data):
    orders_data = []

    if isinstance(data, dict):
        data = [data]

    for order in data:
        customer = order.get("customer", {})

        for product in order.get("products", []):
            row_orders = {
                "order_id": order.get("order_id"),
                "order_date": order.get("order_date"),
                "total_amount": order.get("total_amount"),
                "customer_id": customer.get("customer_id"),
                "customer_name": customer.get("name"),
                "email": customer.get("email"),
                "address": customer.get("address"),
                "product_id": product.get("product_id"),
                "product_name": product.get("name"),
                "category": product.get("category"),
                "price": product.get("price"),
                "quantity": product.get("quantity")
            }
            orders_data.append(row_orders)

    return pd.DataFrame(orders_data)

def lambda_handler(event, context):
    s3 = boto3.client("s3")
    glue = boto3.client("glue")
    output_files = []

    for record in event["Records"]:
        bucket_name = record["s3"]["bucket"]["name"]
        key = urllib.parse.unquote_plus(record["s3"]["object"]["key"])

        if not key.lower().endswith(".json"):
            continue

        response = s3.get_object(Bucket=bucket_name, Key=key)
        content = response["Body"].read().decode("utf-8")
        data = json.loads(content)
        df = flatten(data)

        if df.empty:
            print(f"No products found in file: {key}")
            continue

        parquet_buffer = io.BytesIO()
        df.to_parquet(parquet_buffer, index=False, engine="pyarrow")

        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        key_staging = (
            "target/orders_parquet_datalake/"
            f"orders_ETL_{timestamp}.parquet"
        )

        s3.put_object(
            Bucket=bucket_name,
            Key=key_staging,
            Body=parquet_buffer.getvalue(),
            ContentType="application/octet-stream"
        )
```

```

output_files.append(f"s3://{bucket_name}/{key_staging}")

if output_files:
    try:
        glue.start_crawler(Name="etl_pipeline_crawler")
    except glue.exceptions.CrawlerRunningException:
        print("Glue crawler is already running.")

return {
    "statusCode": 200,
    "message": "JSON flattened and converted to Parquet.",
    "output_files": output_files
}

```

Why rows repeat: If one order contains two products, the flattened output correctly has two rows. Order/customer fields repeat because each row represents a different product line.

9. Create Glue database and crawler role

9.1 Glue database

Open **AWS Glue** → **Data Catalog** → **Databases** → **Add database**. Create:

```
etl_pipeline_db
```

9.2 Crawler IAM role

In IAM, create role: trusted entity = AWS service; service = Glue; role name = **etl-pipeline-crawler-role**. Attach:

```
AWSGlueServiceRole
```

Add this inline policy after replacing the bucket name:

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ListTargetFolder",
      "Effect": "Allow",
      "Action": ["s3:ListBucket"],
      "Resource": "arn:aws:s3:::YOUR-DATA-LAKE-BUCKET",
      "Condition": {
        "StringLike": {
          "s3:prefix": ["target/orders_parquet_datalake/*"]
        }
      }
    },
    {
      "Sid": "ReadParquetFiles",
      "Effect": "Allow",
      "Action": ["s3:GetObject"],
      "Resource": "arn:aws:s3:::YOUR-DATA-LAKE-BUCKET/target/orders_parquet_datalake/*"
    }
  ]
}

```

10. Create Glue crawler

Open **AWS Glue** → **Data Catalog** → **Crawlers** → **Create crawler**, then configure:

Setting	Value
Crawler name	etl_pipeline_crawler
Data source	Amazon S3
S3 path	s3://yourname-etl-data-lake/target/orders_parquet_datalake/
IAM role	etl-pipeline-crawler-role
Target database	etl_pipeline_db
Table prefix	Optional; leave blank for simplicity

The crawler must target only the processed Parquet prefix. Do not crawl source JSON or the Athena results bucket.

11. Add the S3 trigger

Open **Lambda** → **json-to-parquet-lambda** → **Add trigger** and choose S3. Use exactly:

Setting	Value
Bucket	yourname-etl-data-lake
Event type	s3:ObjectCreated:*
Prefix	source/
Suffix	.json
Destination	json-to-parquet-lambda

This filter prevents an infinite loop. JSON uploaded into source/ runs Lambda. The generated .parquet file lands in target/, so it does not match the trigger filter and does not invoke Lambda again.

12. Athena IAM permissions

The IAM identity you use to open Athena and run SQL needs Athena APIs, read access to target Parquet, Glue Catalog read access, and read/write access to the separate Athena results bucket.

For a personal lab, you may attach **AmazonAthenaFullAccess** while learning, but you must still allow S3 write access to the Athena results bucket. For a least-privilege policy, use the following template and replace all placeholders.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "RunAthenaQueries",
      "Effect": "Allow",
      "Action": [
        "athena:StartQueryExecution",
        "athena:GetQueryExecution",
        "athena:GetQueryResults",
        "athena:StopQueryExecution",
        "athena:GetWorkGroup",
        "athena:ListWorkGroups"
      ],
      "Resource": "*"
    },
    {
      "Sid": "ReadProcessedParquet",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:ListBucket"],
      "Resource": [
        "arn:aws:s3:::YOUR-DATA-LAKE-BUCKET",
        "arn:aws:s3:::YOUR-DATA-LAKE-BUCKET/target/orders_parquet_datalake/*"
      ]
    },
    {
      "Sid": "WriteAthenaResults",
      "Effect": "Allow",
      "Action": [
        "s3:GetBucketLocation", "s3:ListBucket", "s3:GetObject",
        "s3:PutObject", "s3:AbortMultipartUpload", "s3:ListMultipartUploadParts"
      ],
      "Resource": [
        "arn:aws:s3:::YOUR-ATHENA-RESULTS-BUCKET",
        "arn:aws:s3:::YOUR-ATHENA-RESULTS-BUCKET/athena-query-results/*"
      ]
    },
    {
      "Sid": "ReadGlueCatalog",
      "Effect": "Allow",
      "Action": [
        "glue:GetDatabase", "glue:GetDatabases", "glue:GetTable",
        "glue:GetTables", "glue:GetPartition", "glue:GetPartitions"
      ],
    }
  ]
}
```

```
"Resource": ""
}
]
}
```

13. Test file

Create a local file called **orders.json** and upload it into `s3://yourname-etl-data-lake/source/`.

```
[
{
  "order_id": 13,
  "order_date": "2024-01-10",
  "total_amount": 255.50,
  "customer": {
    "customer_id": 113,
    "name": "John Doe",
    "email": "johndoe@example.com",
    "address": "121 Main St, Springfield"
  },
  "products": [
    {
      "product_id": "P01",
      "name": "Wireless Mouse",
      "category": "Electronics",
      "price": 25.00,
      "quantity": 2
    },
    {
      "product_id": "P02",
      "name": "Bluetooth Keyboard",
      "category": "Electronics",
      "price": 45.00,
      "quantity": 1
    }
  ]
}
]
```

14. Validate every stage

Stage	Where to check	Expected result
1. Source upload	S3 source/	orders.json exists
2. Lambda invocation	Lambda → Monitor → CloudWatch logs	Successful invocation, no import/access errors
3. Processed output	S3 target/orders_parquet_datalake/	New orders_ETL_YYYYMMDD_HHMMSS.parquet file
4. Crawler	Glue → Crawlers	Status changes Running then Ready
5. Catalog table	Glue → Data Catalog → Tables → etl_pipeline_db	New table with flattened columns
6. Athena query	Athena editor	Rows returned from Parquet table
7. Query output	Separate Athena results bucket	CSV/metadata output appears there

15. Athena queries

Select data source **AwsDataCatalog** and database **etl_pipeline_db**. First list the crawler-created table name:

```
SHOW TABLES;
```

Then replace `YOUR_TABLE_NAME` with the table you see:

```
SELECT *
FROM "AwsDataCatalog"."etl_pipeline_db"."YOUR_TABLE_NAME"
LIMIT 10;
```

Product revenue summary:

```
SELECT
product_name,
```

```
SUM(quantity) AS units_sold,
SUM(price * quantity) AS revenue
FROM "AwsDataCatalog"."etl_pipeline_db"."YOUR_TABLE_NAME"
GROUP BY product_name
ORDER BY revenue DESC;
```

Customer purchase summary:

```
SELECT
customer_name,
email,
COUNT(DISTINCT order_id) AS order_count,
SUM(price * quantity) AS calculated_product_total
FROM "AwsDataCatalog"."etl_pipeline_db"."YOUR_TABLE_NAME"
GROUP BY customer_name, email
ORDER BY calculated_product_total DESC;
```

Find orders where declared total does not equal the summed product lines:

```
SELECT
order_id,
MAX(total_amount) AS declared_order_total,
SUM(price * quantity) AS calculated_product_total,
MAX(total_amount) - SUM(price * quantity) AS difference
FROM "AwsDataCatalog"."etl_pipeline_db"."YOUR_TABLE_NAME"
GROUP BY order_id
HAVING MAX(total_amount) <> SUM(price * quantity);
```

16. Common errors and fixes

Error	Cause	Fix
No output location provided	Athena results location not configured	Set the separate S3 Athena output path in Settings or workgroup
Expected magic number PARquet	CSV/JSON stored inside Parquet table location	Remove non-Parquet files; crawler must scan only target Parquet prefix
mismatched input limit	Ran LIMIT without SELECT	Use SELECT * FROM table LIMIT 10;
No module named pandas	Layer missing/incompatible	Attach matching Amazon Linux layer with pandas
No module named pyarrow	Layer has Pandas but no PyArrow	Rebuild/add layer including pyarrow
AccessDenied: GetObject	Lambda/crawler cannot read S3	Check S3 read policy and exact bucket/prefix
AccessDenied: PutObject	Lambda cannot write target	Allow s3:PutObject on target prefix
AccessDenied: StartCrawler	Lambda lacks Glue permission	Add glue:StartCrawler and glue:GetCrawler
CrawlerRunningException	Several uploads invoke Lambda during a crawl	Catch the exception as shown in code
Lambda triggers repeatedly	S3 trigger catches target output	Set prefix source/ and suffix .json
No table in Athena	Crawler did not find output or failed	Check target Parquet exists; run/check crawler logs

17. Important data-quality note

In the sample file, product lines total 95.00: Wireless Mouse = 25 × 2 = 50; Bluetooth Keyboard = 45 × 1 = 45. The declared total_amount is 255.50, so it differs by 160.50. This is not a pipeline failure; it is a useful data-quality rule you can detect with the Athena mismatch query above.

18. Realistic improvements

- Add **line_total** in Lambda: price × quantity.
- Write partitioned paths such as year=2026/month=09/day=24/ to reduce Athena data scanned for date-filtered queries.
- Use an SQS dead-letter queue or Lambda failure destination for failed S3 events.
- Create CloudWatch alarms for Lambda Errors, Throttles, and Duration.

- Enable S3 Versioning for raw source files.
- For bigger data, use AWS Glue ETL rather than Lambda/Pandas.
- Use AWS Step Functions or EventBridge if you need retries, state tracking, or advanced orchestration.
- Use AWS KMS permissions if either bucket uses customer-managed KMS encryption.

19. Project description

Resume / LinkedIn version: Built a serverless event-driven ETL pipeline on AWS that ingests nested JSON transaction data from Amazon S3. Configured S3 object-created events to invoke AWS Lambda, which flattened customer and product records using Pandas, converted output to Apache Parquet through PyArrow, and stored optimized files in a separate S3 target prefix. Automated schema discovery with AWS Glue Crawler and queried the cataloged dataset through Amazon Athena. Implemented separate raw, processed, and query-result storage locations along with IAM least-privilege access and CloudWatch-based troubleshooting.

20. Cleanup to avoid cost

After you finish the project, delete resources you no longer need:

- Delete S3 test objects, then delete both S3 buckets if not needed. Buckets must be empty before deletion.
- Delete the Lambda function and layer versions if no longer required.
- Delete the Glue crawler and optionally the Glue table/database.
- Check Athena result files in the results bucket.
- Review CloudWatch log retention; set a short retention period for a lab project.
- Remove unused IAM roles/policies only after confirming no other resource uses them.